DolFin Transaction Classification

The purpose of this python notebook is to develop a NLP model that reads the transaction description and then classify the transaction into a category. The reasoning behind using BERT is because it is the current state of the art NLP model avalible and due to our lack of training data, transfer learning would be a practical solution.

Libaries Requried

The following libaries are required for getting the model operational.

```
# Download and install the requied non-standard libaries required for the model
!wget --quiet https://raw.githubusercontent.com/tensorflow/models/master/official/nlp/bert/tokenization.py
!pip install tokenization
!pip install transformers
!pip install tensorflow-hub
```

```
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.25.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (24.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (6.0.1)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2023.12.25)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.31.0)
Requirement already satisfied: tokenizers<0.20,>=0.19 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.19.1)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.4.3)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.66.4)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.19.3->tra
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2024.2
Requirement already satisfied: tensorflow-hub in /usr/local/lib/python3.10/dist-packages (0.16.1)
Requirement already satisfied: numpy>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow-hub) (1.25.2)
Requirement already satisfied: protobuf>=3.19.6 in /usr/local/lib/python3.10/dist-packages (from tensorflow-hub) (3.20.3)
Requirement already satisfied: tf-keras>=2.14.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow-hub) (2.15.1)
Requirement already satisfied: tensorflow<2.16,>=2.15 in /usr/local/lib/python3.10/dist-packages (from tf-keras>=2.14.1->tensorflo
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras>=2
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras
Requirement already satisfied: flatbuffers>=23.5.26 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-ke
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.1
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-ker
Requirement already satisfied: h5py>=2.9.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras>=2.14
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras>
Requirement already satisfied: ml-dtypes~=0.2.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras>
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras>=2.14.1
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras>=2.14
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras>=2.14
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-keras>
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->t
Requirement already satisfied: wrapt<1.15,>=1.11.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-ker
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-ker
Requirement already satisfied: tensorboard<2.16,>=2.15 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf
Requirement already satisfied: tensorflow-estimator<2.16,>=2.15.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16
Requirement already satisfied: keras<2.16,>=2.15.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow<2.16,>=2.15->tf-ker
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from astunparse>=1.6.0->tensorflow<2
Requirement already satisfied: google-auth<3,>=1.6.3 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2.15->ten
Requirement already satisfied: google-auth-oauthlib<2,>=0.5 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2.15->tensorflo
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2.15->tenso
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard<
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2.15->tensorflo
Requirement already satisfied: cachetools<6.0,>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->tens
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->tenso
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->tensorboard<2
Requirement already satisfied: requests-oauthlib>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from google-auth-oauthlib<2,>=
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorboard<2.16
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorboar
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorboar
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.10/dist-packages (from werkzeug>=1.0.1->tensorboard<2.1
Requirement already satisfied: pyasn1<0.7.0,>=0.4.6 in /usr/local/lib/python3.10/dist-packages (from pyasn1-modules>=0.2.1->google
Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.10/dist-packages (from requests-oauthlib>=0.7.0->google-a
```

```
# Processing Utilities
import datetime
import os
import numpy as np
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
# from sklearn.preprocessing import LabelEncoder
from sklearn import preprocessing
import tokenization
from transformers import BertTokenizer

# Natural Language Processing
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from transformers import BertTokenizer, TFBertForSequenceClassification
import nltk
nltk.download('punkt')
nltk.download('stopwords')

# Deep learning
import tensorflow as tf
import tensorflow_hub as hub
from tensorflow import keras
from keras import Model, layers

# Plotting utillities
import matplotlib.pyplot as plt
import seaborn as sns

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

colors = ['#343a40', '#0077bb', '#d9d9d9', '#fafafe', '#abb5be']
sns.set_palette(sns.color_palette(colors))
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Data Labeling

The data is labeled into categories based on a keyword search. Using nested for loops, we itterate through each transaction description field and if a keyword matches then it will assign a class in the label feature. The code can be modified into more classes or have more keywords added easily through ammeding the list, or including additional lists for new classes.

```
# Data location, batch and image size
dataset_dir = '/content/drive/MyDrive/Colab_Notebooks/Projects/DolFin_Transaction_Classification/'
file_name = 'transaction_ut.csv'

# Load the dataset into a dataframe
df = pd.read_csv(dataset_dir + file_name)

# Shuffle the dataframe
df = df.sample(frac=1).reset_index(drop=True)

# Store the column names
labels = list(df.columns)

# Determine the shape of the data
num_samples = df.shape[0]
num_features = df.shape[1]
```

```
# Use keyword search to assign a class for each of the transactions
# Home
keyword_home = ['energy', 'water', 'telecommunications', 'electricity', 'morgage', 'loan']

# Personal
keyword_personal = ['payroll', 'wage', 'wages', 'children', 'fitness', 'education', 'health']

# Good Life
keyword_goodlife = ['atm', 'tran', 'transaction', 'fee', 'holiday', 'travel', 'takeaway']

# Transport
keyword_transport = ['insurance', 'car', 'parking', 'taxi', 'uber', 'toll', 'fuel', 'bike']


labels  = ['home', 'personal','goodlife','transport']

keyword_list = [keyword_home, keyword_personal, keyword_goodlife, keyword_transport]

# Apply the transformation to the dataset
for label, keyword in zip(labels, keyword_list):
    df['class'] = df.apply(lambda row: label if any(str(keyword).lower() in str(row['description']).lower() for keyword in keyword) else

# Display the unique values
df['class'].unique()
```

```
array(['transport', 'goodlife', 'payment', 'home', 'personal', 'transfer'],
      dtype=object)
```

Tokenization

Not all versions of BERT are compatable with TensorFlow, after trial and error it was found that some third party libaries with BERT are not supported by TensorFlow. The version of BERT that is compatable can be loaded from TensorHub, we then assign this model to the bert keras layer. A specific tokenizer needs to be used with the BERT model, for the tokenizer we specify the vocab file and the word casing, then initalise the tokenizer.

```
# Download the bert model from TensorFlow Hub
model_url = 'https://tfhub.dev/tensorflow/bert_en_uncased_L-12_H-768_A-12/2'
# Create a keras layer with the model and set trainable to False
bert_layer = hub.KerasLayer(model_url, trainable=False)

# specify the vocab file for the bert model tokenizer
vocab_file = bert_layer.resolved_object.vocab_file.asset_path.numpy()

# Specify for the tokenizer to change text to lower case
tokenizer_case = bert_layer.resolved_object.do_lower_case.numpy()
# tokenizer = tokenization.FullTokenizer(vocab_file, do_lower_case)

# Create a tokenizer instance
tokenizer = BertTokenizer(vocab_file, tokenizer_case)
```

Preprocessing

The NLP model can not interperate plain text, it does however understand numbers which is done through tokenization. The function written below will convert a string of words into a list containing each individual word as an entry in the list, and then converted into a number representing each word. The list for each entry is then padded, where entris of zeros are added to ensure each list is the same length.

```
y
```

	goodlife	home	payment	personal	transfer	transport
0	False	False	False	True	False	False
1	True	False	False	False	False	False
2	False	False	False	True	False	False
3	False	True	False	False	False	False
4	True	False	False	False	False	False
...	...	...	...	...	...	...
495	True	False	False	False	False	False
496	True	False	False	False	False	False
497	False	False	False	False	False	True
498	False	False	True	False	False	False
499	False	False	False	False	True	False

500 rows × 6 columns

Next steps: Generate code with y View recommended plots

```
# Create a function that encodes the text as tokens for the BERT model
def bert_encode(texts, tokenizer, max_len=512):
    # Initialise the token, mask and segment lists
    all_tokens = []
    all_masks = []
    all_segments = []

    # Itterate through each of the transaction descriptions
    for text in texts:
        # Replace the entry with tokenized version of the entry
        text = tokenizer.tokenize(text)

        # Entries are trunkated to make room for two special tokens
        text = text[:max_len-2]
        # [CLS] is added to the start of the sequence and [SEP] to the end
        input_sequence = ["[CLS]"] + text + ["[SEP]"]
        # Pad the sequence, to ensure that all sequences are the same length
        pad_len = max_len-len(input_sequence)

        # BERT uses segment IDs to distinguish sentences, just set these to 0
        tokens = tokenizer.convert_tokens_to_ids(input_sequence) + [0] * pad_len
        pad_masks = [1] * len(input_sequence) + [0] * pad_len
        segment_ids = [0] * max_len

        # append the tokens, masks and segment IDs into the respective lists
        all_tokens.append(tokens)
        all_masks.append(pad_masks)
        all_segments.append(segment_ids)

    # Return the output of the function
    return np.array(all_tokens), np.array(all_masks), np.array(all_segments)

# Encode the training data using the preprocessing function above
max_len = 250
x = bert_encode(df['description'], tokenizer, max_len=max_len)
y = df['class']
y = pd.get_dummies(train_labels)

# Split the data into training and testing
# x_train, y_train, x_test, y_test = train_test_split(x, y, test_size=0.2, random_state=42)



# # Visualisation of dataset
# def describe_labels(y_train, y_test):
#   fig, ax = plt.subplots(1, 2, figsize=(16,6))
# #   colors = sns.color_palette('deep')
#   colors = ['#343a40', '#0077bb', '#d9d9d9', '#fafafe', '#abb5be']

#   train_unique_labels = np.unique(y_train, return_counts=True)
#   train_labels = train_unique_labels[0]
#   train_label_count = train_unique_labels[1]
#   ax[0].bar(x=train_labels, height=train_label_count, color=colors)
#   ax[0].set_title('Class Count')

#   ax[1].pie(x=train_label_count, labels=train_labels, colors=colors, pctdistance=0.8, autopct='%1.1f%%')
```

```
#   ax[1].set_title('Class Distribution')

#   plt.colorbar
#   plt.tight_layout()
#   plt.show()
#   return None

# describe_labels(y_train, y_test)



# Build a function for generating the BERT classification model
def build_model(bert_layer, max_len=512):
    # The input layers correespond to inputs required by this version of BERT
    input_word_ids = tf.keras.Input(shape=(max_len,), dtype=tf.int32, name="input_word_ids")
    input_mask = tf.keras.Input(shape=(max_len,), dtype=tf.int32, name="input_mask")
    segment_ids = tf.keras.Input(shape=(max_len,), dtype=tf.int32, name="segment_ids")

    # The pooled output is the BERT model layer which takes the three input layers
    pooled_output, sequence_output = bert_layer([input_word_ids, input_mask, segment_ids])

    # The classifier output layer is used to extract the first token from the sequence
    # which is expected to be the [CLS] token added prior
    classifier_output = sequence_output[:, 0, :]

    # Create some dense layers for the classification portion of the model
    # Dropout layers are used to make the model more generalised.
    layer = tf.keras.layers.Dense(64, activation='relu')(classifier_output)
    layer = tf.keras.layers.Dropout(0.2)(layer)
    layer = tf.keras.layers.Dense(32, activation='relu')(layer)
    layer = tf.keras.layers.Dropout(0.2)(layer)
    output_layer = tf.keras.layers.Dense(6, activation='softmax')(layer)

    # Build the model based on the code we developed above and compile
    model = tf.keras.models.Model(inputs=[input_word_ids, input_mask, segment_ids], outputs=output_layer)
    model.compile(tf.keras.optimizers.Adam(lr=2e-5), loss='categorical_crossentropy', metrics=['accuracy'])

    # Return the model
    return model

# Create a model instance for our transaction classification
model = build_model(bert_layer, max_len=max_len)
# Display a summary of the model
model.summary()
```

```
WARNING:absl:`lr` is deprecated in Keras optimizer, please use `learning_rate` or use the legacy optimizer, e.g.,tf.keras.optimizers
Model: "model_4"
__________________________________________________________________________________________________
 Layer (type)                Output Shape              Param #   Connected to
==================================================================================================
 input_word_ids (InputLayer  [(None, 250)]             0         []
 )

 input_mask (InputLayer)     [(None, 250)]             0         []

 segment_ids (InputLayer)    [(None, 250)]             0         []

 keras_layer_6 (KerasLayer)  [(None, 768),             1094822   ['input_word_ids[0][0]',
                              (None, 250, 768)]        41         'input_mask[0][0]',
                                                                  'segment_ids[0][0]']

 tf.__operators__.getitem_4  (None, 768)               0         ['keras_layer_6[0][1]']
  (SlicingOpLambda)

 dense_12 (Dense)            (None, 64)                49216     ['tf.__operators__.getitem_4[0
                                                                 ][0]']

 dropout_8 (Dropout)         (None, 64)                0         ['dense_12[0][0]']

 dense_13 (Dense)            (None, 32)                2080      ['dropout_8[0][0]']

 dropout_9 (Dropout)         (None, 32)                0         ['dense_13[0][0]']

 dense_14 (Dense)            (None, 6)                 198       ['dropout_9[0][0]']

==================================================================================================
Total params: 109533735 (417.84 MB)
Trainable params: 51494 (201.15 KB)
Non-trainable params: 109482241 (417.64 MB)
__________________________________________________________________________________________________
```

Training Model

Training the BERT model on the DolFin classification data is the same process as with standard TensorFlow classification models. I have included checkpointing and earlystopping into the callbacks to ensure that we have the best model and that it does not overfit the training data.

```
# Create our tensorboard callbacks
checkpoint = tf.keras.callbacks.ModelCheckpoint('model.h5', monitor='val_accuracy', save_best_only=True, verbose=1)
earlystopping = tf.keras.callbacks.EarlyStopping(monitor='val_accuracy', patience=2, verbose=1)
tensorflow_callbacks = [checkpoint, earlystopping]

# Specify the number of epoch, batch size and validation split
num_epochs = 10
batch_size = 32
val_split = 0.2

# Train the NLP model on the data and store its performance
history = model.fit(
    train_input, train_labels,
    validation_split=val_split,
    epochs=num_epochs,
    callbacks=tensorflow_callbacks,
    batch_size=batch_size,
    verbose=1
)

# Save a copy of the model
model.save_weights('BERT_model.h5')
```

```
Epoch 1/10
13/13 [==============================] - ETA: 0s - loss: 1.4922 - accuracy: 0.4200
Epoch 1: val_accuracy improved from -inf to 0.72000, saving model to model.h5
/usr/local/lib/python3.10/dist-packages/keras/src/engine/training.py:3103: UserWarning: You are saving your model as an HDF5 file vi
  saving_api.save_model(
13/13 [==============================] - 12s 379ms/step - loss: 1.4922 - accuracy: 0.4200 - val_loss: 0.9826 - val_accuracy: 0.7200
Epoch 2/10
13/13 [==============================] - ETA: 0s - loss: 0.8568 - accuracy: 0.7400
Epoch 2: val_accuracy improved from 0.72000 to 0.94000, saving model to model.h5
13/13 [==============================] - 4s 310ms/step - loss: 0.8568 - accuracy: 0.7400 - val_loss: 0.5058 - val_accuracy: 0.9400
Epoch 3/10
13/13 [==============================] - ETA: 0s - loss: 0.5139 - accuracy: 0.8875
Epoch 3: val_accuracy improved from 0.94000 to 1.00000, saving model to model.h5
13/13 [==============================] - 4s 295ms/step - loss: 0.5139 - accuracy: 0.8875 - val_loss: 0.2347 - val_accuracy: 1.0000
Epoch 4/10
13/13 [==============================] - ETA: 0s - loss: 0.3267 - accuracy: 0.9300
Epoch 4: val_accuracy did not improve from 1.00000
13/13 [==============================] - 3s 222ms/step - loss: 0.3267 - accuracy: 0.9300 - val_loss: 0.1293 - val_accuracy: 1.0000
Epoch 5/10
13/13 [==============================] - ETA: 0s - loss: 0.2092 - accuracy: 0.9575
Epoch 5: val_accuracy did not improve from 1.00000
13/13 [==============================] - 3s 222ms/step - loss: 0.2092 - accuracy: 0.9575 - val_loss: 0.0633 - val_accuracy: 1.0000
Epoch 5: early stopping
```

```
# Create a function to display the training accuracy and loss
def plt_accuracy_loss(history):
    # Plot the training history
    accuracy = history.history['accuracy']
    loss = history.history['loss']
    epochs = range(len(accuracy))

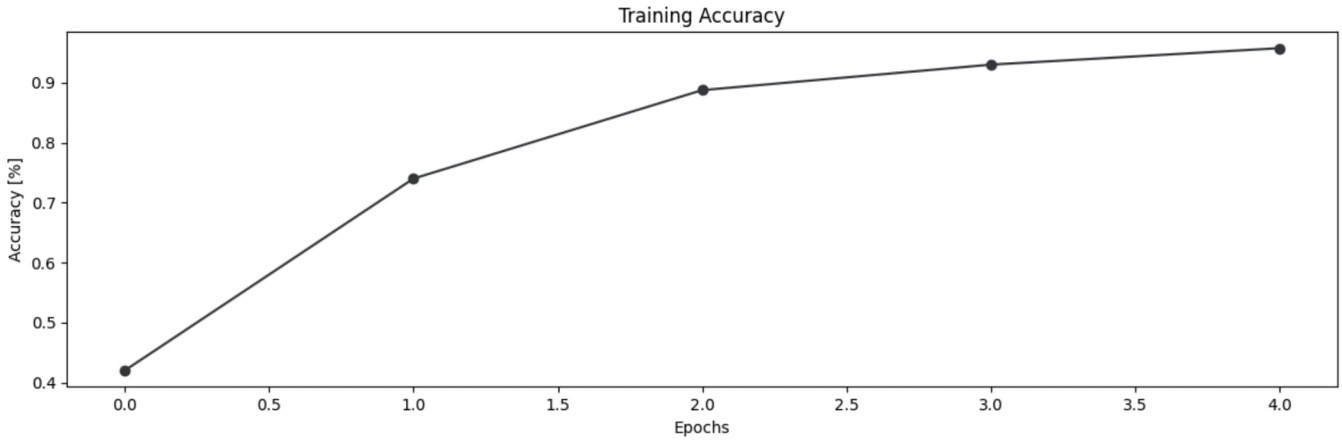
    figure, ax = plt.subplots(2, 1, figsize=(12, 8))

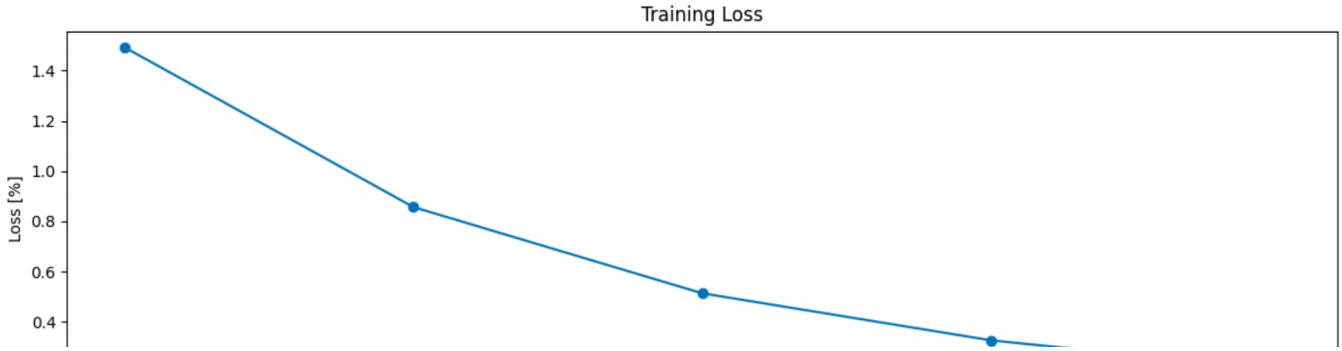
    ax[0].plot(epochs, accuracy, '-o', color=colors[0])
    ax[0].set_title('Training Accuracy')
    ax[0].set_xlabel('Epochs')
    ax[0].set_ylabel('Accuracy [%]')

    ax[1].plot(epochs, loss, '-o', color=colors[1])
    ax[1].set_title('Training Loss')
    ax[1].set_xlabel('Epochs')
    ax[1].set_ylabel('Loss [%]')

    plt.tight_layout()
    plt.show()

plt_accuracy_loss(history)
```


Training Accuracy
Accuracy [%]
0.9
0.8
0.7
0.6
0.5
0.4
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epochs


Training Loss
Loss [%]
1.4
1.2
1.0
0.8
0.6
0.4